

Logiska operatorer

Den allmänna formen för en `if`-sats är ju:

```
if ( villkor )  
    sats;
```

Delen *villkor* har i exemplen ovan varit ett enda uttryck, men C++ tillåter att flera uttryck grupperas samman med hjälp av logiska operatorer. På detta sätt kan man uttrycka förhållanden som *och*, *eller* och *inte*. Operatorerna är i C++ `&&`, `||` respektive `!`.

Varning
Blanda inte ihop de logiska operatorerna <code>&&</code> och <code> </code> med de binära operatorerna <code>&</code> och <code> </code> . I många fall är det helt korrekt C++ att använda de binära operatorerna i t.ex. <code>if</code> -satser, och kompilatorn kommer att godkänna det, även om det inte är vad som avses

Ett kort exempel på hur operatoren `&&` kan användas för att kombinera två uttryck:

```
float Rot;  
cin >> Rot;  
// är Rot inom ett litet intervall från Pi?  
if ( Rot > 3.14158 && Rot < 3.14160 ) {  
    // talet är väldigt nära Pi  
    cout << "Talet " << Rot << " är nära talet Pi." << endl;  
}
```

Här kommer `if`-satsens kropp att utföras om och endast om `Rot` uppfyller båda kraven, det räcker inte med endast det ena. För att kolla om ett uttryck *eller* ett annat gäller illustreras av följande exempel. Här visas även hur flera än två uttryck kan kombinera tillsammans.

```
char Menyval;  
cin >> Menyval;  
// skall foo utföras?  
if ( Menyval == 's' || Menyval == 'S' || Menyval == '1' ) {  
    // jep, utför foo  
    foo ();  
}
```

Operatoren `!` används för att negera en utsaga. Ett uttryck för *olika med* kan således skrivas:

```
if ( ! Ar == 1972 ) {  
    ...  
}
```

Ovannämnda exempel kunde mera lättläst skrivas `Ar != 1972`. När man konstruerar logiska uttryck bör man tänka på operatorernas *precedens* (se [avsnittet Precedens i Kapitel 3](#)) för att undvika uttryck som evalueras på fel sätt. Är man inte säker på precedensen kan man kombinera uttryck med parenteser (se [avsnittet Kombinera uttryck](#)).

Lazy evaluation

C++ använder sig av en finess som kallas *lazy evaluation*. Detta innebär att då sanningsvärdet för en utsaga beräknas evalueras uttryck från vänster till höger ända tills utsagans värde är klart. Om utsagans värde klarnar för hela uttrycket evaluerats ignoreras resten. I en allmän form med operatoren `&&` kan det se ut:

```
if ( A && B && C && D ) {  
    ...  
}
```

Antag att `A`, `B`, `C` och `D` är uttryck av något slag. Eftersom `&&` sammanbinder dem alla måste alla uttryck evalueras till `true` för att `if`-satsen skall utföras. Om t.ex. `B` evalueras till `false` kommer ju hela uttrycket att få värdet `false`, och det är ingen ide att evaluera `C` och `D`. I ett ekvivalent exempel, men med `&&` ersatt med `||` kommer evalueringen att stanna så fort ett av uttrycken evalueras till `true`, eftersom hela uttrycket då måste evalueras till `true`.

Ett bra exempel på hur *lazy evaluation* ofta används är:

```
if ( X != 0 && 4/X < 3 ) {  
    ...  
}
```

Här kan vi ju i misstag dividera med 0 om inte kontrollen `X != 0` funnits. Om `X` har värdet 0 kommer hela uttrycket i vilket fall som helst att få värdet `false`, så `4/X < 3` evalueras aldrig.

Kombinera uttryck

För att kombinera uttryck tillsammans kan man använda sig av normala parenteser, precis som i logiken. Om man t.ex. vill att uttrycket `A` eller `B` skall gälla, och uttrycken `C` eller `D`, kan man skriva det på följande sätt:

```
if ( ( A || B ) && ( C || D ) ) {  
    ...  
}
```

Man bör komma ihåg att operatoren `&&` har högre precedens än `||`, så ovanstående hade utan parenteser egentligen tolkats på följande sätt:

```
if ( A || (B && C) || D ) {  
    ...  
}
```

Alltså inte exakt vad som avsågs. Parenteser evalueras först, börjande från den innersta parentesen, precis enligt normala precedensregler. Det kan vara värt att kontrollera sina logiska uttryck om man har ett program som beter sig konstigt.

[Förutgående](#)

`if-else-satser`

[Hem](#)

[Upp](#)

[Nästa](#)

`switch-uttryck`